

01 · AND / OR / XOR

템플릿에서 비트스트림까지

VS Code 사전 시뮬레이션 → Vivado 2026.1

1. 공개 템플릿을 clone하고 새 창에서 연다.
2. RTL·TB·XDC를 직접 작성하고 네 입력 조합을 검사한다.
3. 같은 소스를 Vivado에 등록하고 다시 시뮬레이션한다.
4. Spartan-7용 비트스트림을 생성한다.

작성일 2026. 09. 10. 이해리

01 · 실습 목차

1부 · VS Code

도구 확인·템플릿 clone·새 창 열기

RTL·테스트벤치·예상값

실행 task·로그·VaporView·실험 전 레포트

2부 · Vivado 2026.1

프로젝트 생성·소스 임포트·top 지정

Behavioral Simulation·파형 비교

핀 확인·합성·구현·비트스트림

보드 연결·사진·영상·GitHub·실험 후 레포트

좌하단 목차는 이 PDF의 2쪽으로 돌아온다.

시뮬레이션 도구 확인

Git·Python 3.10 이상·VS Code·Icarus Verilog를 설치한다. Windows PowerShell에서 한 줄씩 실행한다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

설치 경로를 PATH에 추가했다면 VS Code를 다시 연다. Linux·macOS·WSL은 python 대신 python3를 사용한다.

v2.0.0 템플릿을 새 이름으로 clone

아래는 Windows PowerShell 명령이다. 첫 줄 끝의 백틱(backtick)은 명령을 다음 줄로 이어 준다.

```
git clone --branch v2.0.0 `
https://github.com/Glaysia/fpga-lab-template.git lab1_01_logic_gates
cd lab1_01_logic_gates
git switch -c main
code LAB1.code-workspace
```

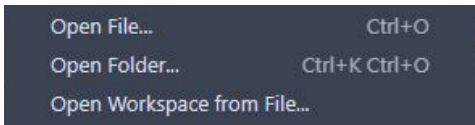
태그로 같은 시각 파일을 받는다. git switch는 태그에서 학생 작업용 main 브랜치를 만든다. 다음 회로는 목적지 폴더 이름을 바꾼다.

File → New Window

VS Code 상단 File → New Window. Ctrl+Shift+N도 같은 동작이다.



새 창의 File → Open Workspace from File...을 누른다.



프로젝트 하나의 workspace 열기

1. 방금 clone한 lab1_01_logic_gates 폴더를 연다.
2. LAB1.code-workspace를 선택하고 Open. 모든 실습의 workspace 파일명은 같다.
3. Explorer에 PROJECT 하나와 src.sim.constraints.tools 폴더가 보이는지 확인한다.
4. src/design.v.sim/tb_design.v.constraints/pins.xdc는 아직 주석뿐이다.
5. simulation.json은 실행할 RTL.TB 파일과 TB top을 지정하는 설정이다.

추천 확장 설치

1. 왼쪽 Extensions 아이콘을 누른다.
2. 검색창에 @recommended를 입력한다.
3. slang의 Install: Verilog/SystemVerilog 편집·진단.
4. VaporView의 Install: VCD 파형 확인.
5. vscode-pdf의 Install: 코드 옆에서 PDF 교안 열기.

WSL 사용자는 WSL 창에서 필요할 경우 Install in WSL을 누른다. 확장 설치와 Python·Icarus 설치의 별개다.

src에 RTL 파일 만들기

1. Explorer에서 src 왼쪽 화살표를 눌러 펼친다.
 2. design.v 우클릭 → Rename → logic_gate.v 입력.
 3. 파일을 두 번 눌러 열고 뒤의 코드 이미지를 보며 전체 코드를 입력한다.
 4. 추가 RTL이 있으면 src 우클릭 → New File로 만든다.
 5. 전체 RTL 파일 목록은 다음 장의 src 표를 따른다.
 6. File → Save All로 모든 파일을 저장한다.
- 설계 모듈명: logic_gate. 파일명과 module 이름을 구분한다.

sim에 테스트벤치 만들기

1. sim을 펼치고 tb_design.v 우클릭 → Rename.
2. 새 파일명: tb_logic_gate_modern.sv
3. 뒤의 TB 코드 이미지에 있는 선언·DUT 연결·입력 자극·예상값
검사를 직접 작성한다.
4. wave.vcd를 만드는 dumpfile·dumpvars와 종료 조건까지 작성한다.
5. TB 모듈명: tb_logic_gate_modern
6. File → Save All. TB를 합성용 RTL 폴더에 넣지 않는다.

작성할 RTL 파일 목록

폴더	파일명
src	logic_gate.v

각 파일을 New File로 만들고 코드 이미지의 전체 내용을 작성한다.
파일마다 module 선언과 endmodule을 확인한다.

simulation.json을 내 파일에 맞추기

Explorer에서 simulation.json을 두 번 눌러 연다.

sources 배열: 앞의 모든 RTL 파일명 앞에 src/를 붙여 등록한다.

한 파일 예: "sources": ["src/logic_gate.v"]

testbench: sim/tb_logic_gate_modern.sv

simulation_top: tb_logic_gate_modern

sources의 각 경로와 testbench·simulation_top 값은 큰따옴표로 감싼다.
여러 경로는 쉼표로 구분한다.

파일명·확장자·괄호·쉼표를 확인하고 Save All.

실행 전 예상값

a	b	x AND	y OR	z XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

이번 최신 테스트는 $00 \rightarrow 01 \rightarrow 10 \rightarrow 11$, 각 10ns다.
OR는 하나 이상 1, XOR는 서로 다른 입력일 때 1이다.
이 표를 실험 전 레포트에 먼저 작성한다.

RTL 코드를 직접 입력한다

Explorer에서 PROJECT → src → logic_gate.v를 두 번 누른다.

```
1 module logic_gate(input wire a,b, output wire x,y,z);  
2     assign x = a & b;  
3     assign y = a | b;  
4     assign z = a ^ b;  
5 endmodule
```

입력 a·b와 출력 x·y·z를 확인한다.

assign 세 줄이 AND·OR·XOR 조합회로를 연결한다.

이 이미지의 전체 코드를 직접 입력하고 File → Save All을 누른다.

테스트벤치 직접 입력 · 1-12행

PROJECT → sim → tb_logic_gate_modern.sv를 연다.

```
1 | timescale 1ns/1ps
2 | module tb_logic_gate_modern;
3 |     reg a,b; wire x,y,z; logic_gate dut(a,b,x,y,z);
4 |     integer n,checked=0;
5 |     reg [2:0] expected;
6 |     initial begin
7 |         $dumpfile("wave.vcd"); $dumpvars(0,tb_logic_gate_modern);
8 |
9 |         for(n=0;n<4;n=n+1) begin
10 |             {a,b}=n;
11 |             expected={ (a & b), (a | b), (a ^ b) };
12 |             #10;
```

1행의 첫 문자는 backtick이다. dut는 검사할 논리 게이트다.

{a,b}=n으로 네 입력을 만들고 10ns씩 기다린다.

여기서 끝내지 말고 다음 장의 13-22행을 이어 입력한다.

테스트벤치 직접 입력 · 13-22행

같은 파일에서 앞 장의 12행 다음에 이어 입력한다.

```
13         if ({x,y,z} != expected)
14             $fatal(1,"FAIL logic_gate vector=%0d expected=%h
actual=%h",n,expected,{x,y,z});
15             checked=checked+1;
16         end
17         if(checked!=4) $fatal(1,"Incomplete test");
18         $display("LAB1_PASS logic_gate cases=%0d",checked);
19         $finish;
20     end
21     initial begin #100000; $fatal(1,"Watchdog"); end
22 endmodule
```

14행은 화면에서만 두 줄로 접혔다. 문자열 중간에 Enter를 넣지 않는다.
기대값과 실제 출력이 다르면 \$fatal로 실패한다.
검사 수·PASS·종료·watchdog·마지막 endmodule까지 저장한다.

시간·완료 조건 확인

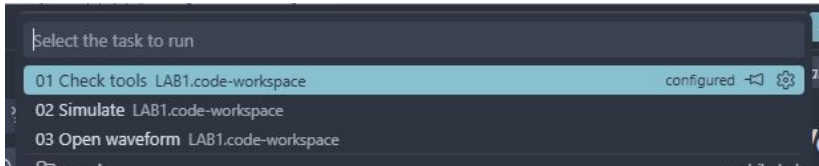
- `timescale 1ns/1ps`: 시간 단위는 1ns, 정밀도는 1ps.
- `#10`: 입력을 준 뒤 10ns 동안 기다린다.
- 4개를 모두 검사하면 `LAB1_PASS logic_gate cases=4`.
- `$finish`로 40ns에서 종료한다.
- `wave.vcd`: 시뮬레이션 파형 원본.

기존 레거시 교안의 200ns 간격과 구분한다.

최신 VS Code와 최신 Vivado는 이 동일한 테스트벤치를 사용한다.

저장하고 작업 메뉴 열기

코드를 수정했다면 File → Save All. 다음은 Terminal → Run Task....



창이 좁으면 상단 ...에 Terminal 메뉴가 있다.
먼저 01 Check tools, 다음에 02 Simulate를 선택한다.
Ctrl+Shift+B 기본 작업도 02 Simulate다.

세 작업의 의미

01 Check tools

설치·버전 검사

02 Simulate

Icarus로 내 RTL·TB 실행

03 Open waveform

생성한 VCD를 VS Code로 열기

작업이 보이지 않으면 LAB1.code-workspace를 열었는지 확인한다.
실행 스크립트는 프로젝트의 tools/lab1.py, 회로 설정은
simulation.json이다.

PASS와 종료 시각 확인

터미널 또는 `build/sim/run-.../simulation.log`를 연다.

실행 명령에 `src/logic_gate.v`와 `sim/tb_logic_gate_modern.sv`가 포함되는지 확인한다. `SIMULATED`는 새 파형 생성 완료를 뜻하며, TB의 정답 검사와 구분한다.

`LAB1_PASS logic_gate cases=4`와 `40ns` 종료를 확인한다.

실패하면 이번 실행의 `compile.log·simulation.log`를 읽는다.

실행기는 실패 후 예전 VCD를 새 성공 결과로 남기지 않는다.

VCD를 VaporView로 열기

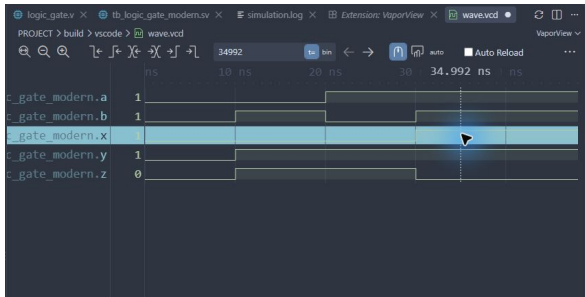
03 Open waveform을 실행하거나 build/sim/wave.vcd를 연다.

글자로 열리면 파일 탭을 마우스 오른쪽 클릭 →
Reopen Editor With... → VaporView.

1. 빈 파형 화면의 Netlist View를 누른다.
2. tb_logic_gate_modern을 펼친다.
3. a, b, x, y, z를 각각 두 번 눌러 추가한다.
4. 파형 도구 모음의 Zoom to Fit로 전체 시간을 맞춘다.
5. 아래 Time Units를 ns로 맞춘다.

파형에서 실제 값을 읽는다

시간 구간 안쪽을 클릭하여 세로 커서와 신호 값을 확인한다.



0-10ns: 00 / 10-20ns: 01 / 20-30ns: 10 / 30-40ns: 11.

마지막 구간에서 AND=1, OR=1, XOR=0인지 확인한다.

입력이 전환되는 경계 대신 구간의 중간을 읽는다.

XOR를 바꾸고 실패를 확인한다

1. src/logic_gate.v에서 XOR 연산자 ^를 OR 연산자 |로 바꾼다.
 2. 어떤 입력에서 결과가 달라질지 진리표에 먼저 표시한다.
 3. File → Save All → Terminal → Run Task... → 02 Simulate.
 4. 실패 로그에서 입력 벡터·expected·actual을 읽고 예상한 오류인지 설명한다.
 5. 연산자를 ^로 복구하고 저장한 뒤 02를 다시 실행한다.
 6. cases=4 통과와 새 파형을 확인하고 실패·복구 증빙을 남긴다.
- TB의 예상값은 함께 바꾸지 않는다. 설계만 바꿨을 때 검사가 오류를 찾아야 한다.

constraints에 XDC 직접 작성

1. constraints를 펼치고 pins.xdc 우클릭 → Rename.
2. 파일명: logic_gate.xdc
3. 핀 표와 코드 이미지를 보며 PACKAGE_PIN·IOSTANDARD·get_ports를 입력한다.
4. get_ports의 이름과 비트 번호를 자신이 작성한 RTL의 포트와 대조한다.
5. File → Save All. Vivado 또는 CLI 구현에는 이 파일을 직접 가져간다.

XDC는 Icarus 기능 시뮬레이션에서 사용하지 않는다. 파형 통과만으로 핀 배치까지 검증됐다고 판단하지 않는다.

XDC 전체 내용 · 1-12행

constraints/logic_gate.xdc를 열고 아래 내용을 직접 입력한다.

```
1  # Derived from vendor archive; see legacy provenance.
2  set_property PACKAGE_PIN N8 [get_ports b]
3  set_property PACKAGE_PIN L4 [get_ports x]
4  set_property PACKAGE_PIN M4 [get_ports y]
5  set_property PACKAGE_PIN M2 [get_ports z]
6  set_property IOSTANDARD LVCMOS33 [get_ports b]
7  set_property IOSTANDARD LVCMOS33 [get_ports a]
8  set_property IOSTANDARD LVCMOS33 [get_ports x]
9  set_property IOSTANDARD LVCMOS33 [get_ports y]
10 set_property IOSTANDARD LVCMOS33 [get_ports z]
11
12 set_property PACKAGE_PIN K4 [get_ports a]
```

a=K4, b=N8, x=L4, y=M4, z=M2이며 다섯 포트 모두 LVCMOS33이다.

마지막 a 핀 지정까지 입력하고 Save All. Icarus는 이 파일을 읽지 않으므로 Vivado에서 핀도 확인한다.

실험 전 레포트

1. 회로 목적·입출력·진리표·RTL 세 연결을 설명한다.
2. 템플릿 경로, 도구 버전, 실행 task, 코드 커밋을 기록한다.
3. 테스트벤치의 네 입력·기대값·10ns 간격·40ns 종료를 설명한다.
4. 자신의 PASS 로그와 $a \cdot b \cdot x \cdot y \cdot z$ 파형을 캡처한다.
5. 네 시간 구간을 표로 정리하고 OR와 XOR 차이를 해석한다.
6. 오류가 있었다면 원인·수정·저장·재실행 결과를 적는다.

저장소 소스를 복사해서 사용해도 된다. 구조와 결과를 설명해야 한다.
교재 캡처를 자신의 실행 결과로 제출하지 않는다.

Vivado에서 직접 이어서 실습

VS Code에서 파형과 사전 레포트를 확인한 뒤 Vivado 2026.1을 연다.

이제부터는 Vivado의 메뉴와 버튼을 직접 누른다.

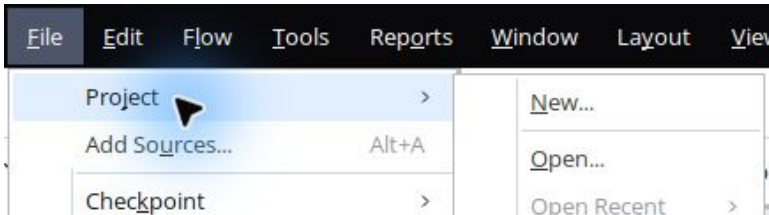
프로젝트 생성, 파일 등록, 시뮬레이션, 합성, 구현, 비트스트림 순서다.

방금 직접 작성한 `src.sim.constraints`의 파일을 가져온다.

파일 선택 화면의 경로는 자신의 `lab1_01_logic_gates` 폴더로 바꾼다.

새 프로젝트 메뉴

상단 File → Project → New...를 선택한다.



기존 프로젝트가 열려 있어도 같은 메뉴에서 시작할 수 있다.
처음 시작 화면에서는 Create Project를 선택해도 된다.
New Project 안내 화면이 먼저 나오면 Next를 누른다.

프로젝트 이름과 저장 위치

Project name은 logic_gate로 입력한다.

Project name: ×

Project location: × ...

☐ Create project subdirectory

Project will be created at: C:/Users/UOS/Documents/fpga lab review/vivado_2026_1/01_logic_gates/vivado

[Back](#) [Next](#)

Project location: 자신의 lab1_01_logic_gates 폴더 / vivado.
Create project subdirectory를 해제한다. 생성 위치를 확인하고 Next.
캡처의 사용자 폴더명을 그대로 입력하지 않는다.

RTL Project 선택

RTL Project와 Do not specify sources at this time을 선택한다.

- ☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL
- ☒ **Do not specify sources at this time**
- ☐ Project is an extensible Vitis platform

Back

Next

파일은 빈 프로젝트를 만든 뒤 종류별로 추가한다.

Project is an extensible Vitis platform은 선택하지 않는다. Next.

FPGA part 검색과 선택

Parts 탭의 검색란에 xc7s75fgga484-1을 입력한다.

Part	I/O Pin Count	Available IOBs	LUT Elements	F
xc7s75fgga484-1	484	338	48000	9
xc7s75fgga484-1IL	484	338	48000	9
xc7s75fgga484-1Q	484	338	48000	9

검색 결과에서 정확히 xc7s75fgga484-1인 첫 행을 선택한다.
뒤에 IL 또는 Q가 붙은 다른 part와 구분하고 Next.

생성 요약 확인

이름과 부품이 맞으면 Finish를 누른다.

New Project Summary

- A new RTL project named 'logic_gate' will be created.
- The default part and product family for the new project:
Default Part: xc7s75fgga484-1
Family: Spartan-7
Package: fgga484
Speed Grade: -1

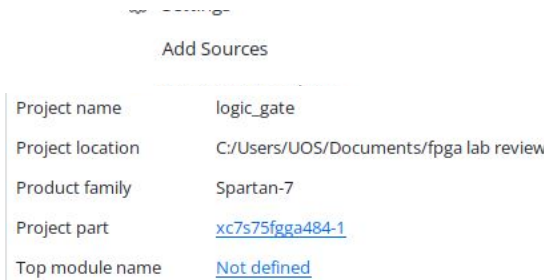


Family: Spartan-7 / Package: fgga484 / Speed Grade: -1.

기존 프로젝트를 닫을지 물으면 자신의 작업을 저장한 뒤 진행한다.

빈 프로젝트 화면 읽기

Sources와 Project Summary의 위치를 먼저 확인한다.

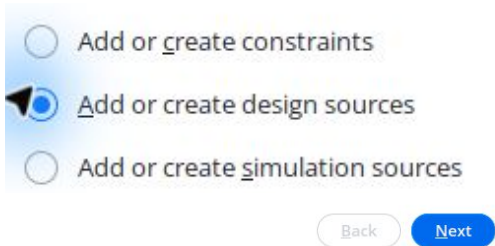


Add Sources	
Project name	logic_gate
Project location	C:/Users/UOS/Documents/fpga lab review
Product family	Spartan-7
Project part	<u>xc7s75fgga484-1</u>
Top module name	<u>Not defined</u>

아직 파일이 없으므로 Top module name이 Not defined인 것은 정상이다.
왼쪽 Add Sources에서 RTL, 테스트벤치, 핀 제약을 차례로 등록한다.

설계 RTL 종류 선택

Add Sources → Add or create design sources를 선택한다.



실제 FPGA 회로는 Design Sources에 넣는다.
다른 종류를 선택하지 않았는지 확인한 뒤 Next.

기존 파일은 Add Files

가운데 Add Files를 누른다.

Add Files

Add Directories

Create File

VS Code에서 직접 작성한 파일을 선택한다.
다음 창에서 src/logic_gate.v를 선택한다.

논리 게이트 RTL 선택

File name에서 자신의 src/logic_gate.v를 지정한다.

File name C:/Users/UOS/Documents/fpga lab review/common/rtl/logic_gate.v
Files of type Design Source Files (.vhd, vhdl, vhf, vhdp, vho, v, vf, verilog, vr, vg, vb, tf, vlog, vp, vm, veo, svo, sveo, vh, h, svh, vhp, svhp, edn, edf, edif, ngc,

Open

폴더를 찾아 선택하거나 파일의 전체 경로를 입력하고 Open.
확장자는 .v이며, 테스트벤치 .sv 파일을 고르는 단계가 아니다.

RTL 등록과 Copy 옵션

목록에 logic_gate.v가 있는지 확인한다.

	Index	Name	Library	Version
	1	logic_gate.v	xil_defaultlib	N/A

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories

Back

Finish

Copy sources into project는 해제한다.

VS Code에서 편집한 같은 원본을 Vivado도 참조하도록 Finish를 누른다.

테스트벤치 종류 선택

Add Sources를 다시 누른다.

- ☐ Add or greate constraints
- ☐ Add or create design sources
- ☒  Add or create simulation sources

Back

Next

이번에는 Add or create simulation sources를 선택하고 Next.
테스트벤치는 실제 FPGA에 합성할 회로가 아니다.

테스트벤치 파일 선택

Add Files에서 sim/tb_logic_gate_modern.sv를 고른다.

File name	C:/Users/UOS/Documents/fpga lab review/common/tb/tb_logic_gate_modern.sv
Files of type	Design Source Files (.vhd, vhdl, vhfl, vhdn, vho, v, vf, verilog, vr, vg, vb, tf, vlog, vp, vm, veo, svo, sveo, vh, h, svh, vhp, svhp, edn, edf, edif, ngc, sv, svp, bxn, bmm,

OK

VS Code에서 네 입력을 검사한 동일한 파일이다.
File name과 확장자 .sv를 확인하고 OK 또는 Open.

시뮬레이션 등록 옵션

Specify simulation set은 sim_1로 둔다.

Specify simulation set: 📁 sim_1 ▼

	Index	Name	Library	Version
●	1	tb_logic_gate_modern.sv	xil_defaultlib	N/A

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories
- ☒ Include all design sources for simulation

Back Finish

Copy sources into project는 해제한다.

Include all design sources for simulation은 체크한 뒤 Finish.

핀 제약 종류 선택

다시 Add Sources → Add or create constraints.

- ☒ Add or create constraints
- ☐ Add or create design sources
- ☐ Add or create simulation sources

Back

Next

XDC는 포트와 FPGA 핀, 전기 규격을 연결한다.
Design Sources 또는 Simulation Sources에 넣지 않고 Next.

XDC 파일 선택

Add Files에서 constraints/logic_gate.xdc를 고른다.

File name C:/Users/UOS/Documents/fpga lab review/common/constraints/logic_gate.xdc
Files of type Design Constraint Files (.sdc, xdc)

OK

File name과 확장자 .xdc를 확인하고 OK.
이 XDC는 이번 강의의 원본 장비를 위한 핀 매핑이다.

제약 파일 등록 완료

목록에 logic_gate.xdc가 있는지 확인한다.

Constraint File	Location
logic_gate.xdc	C:\Users\UOS\Documents\fpga lab review\common\constraints

☐ Copy constraints files into project

Back

Finish

Copy constraints files into project를 해제하고 Finish.
Constraints → constrs_1 아래에서 등록된 파일을 확인한다.

세 종류와 두 top 확인

Sources의 화살표를 눌러 계층을 펼친다.



Design Sources: logic_gate / Constraints: logic_gate.xdc.
Simulation Sources → sim_1: tb_logic_gate_modern.
테스트벤치 아래 dut : logic_gate가 연결되어 있어야 한다.

Set as Top은 언제 누르나

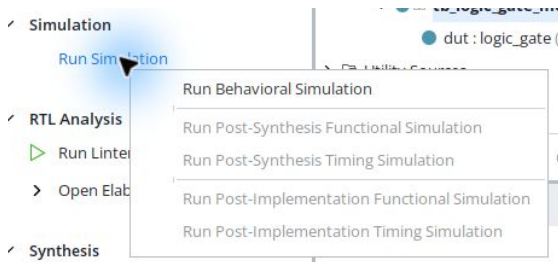
시뮬레이션의 가장 위 모듈은 tb_logic_gate_modern이다.



올바른 모듈이 굵게 보이면 이미 top이므로 그대로 둔다.
다른 모듈이 top이면 테스트벤치를 우클릭 → Set as Top.
설계 top은 logic_gate로 유지한다.

Behavioral Simulation 실행

Run Simulation → Run Behavioral Simulation을 선택한다.

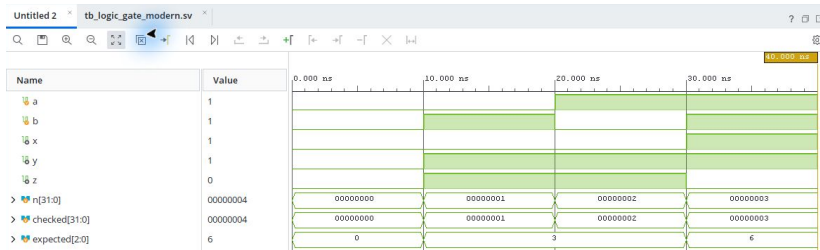


컴파일과 elaboration이 끝날 때까지 기다린다.

실패하면 Messages 또는 Tcl Console의 오류를 먼저 확인한다.

파형 탭과 Zoom Fit

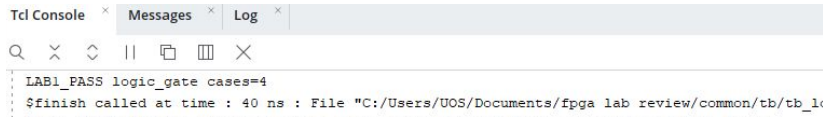
Untitled 파형 탭을 열고 탭을 두 번 눌러 크게 볼 수 있다.



파형 도구 모음의 네 방향 화살표 Zoom Fit를 누른다.
a, b, x, y, z의 0~40ns 네 구간을 VS Code 파형과 비교한다.

PASS와 실제 종료 시각

아래 Tcl Console 탭을 눌러 실행 결과를 확인한다.



The screenshot shows a software interface with three tabs: 'Tcl Console', 'Messages', and 'Log'. The 'Tcl Console' tab is active. Below the tabs is a toolbar with icons for search, close, zoom, and other functions. The console area displays the following text:

```
LAB1_PASS logic_gate cases=4
$finish called at time : 40 ns : File "C:/Users/UOS/Documents/fpga lab review/common/tb/tb_1<
```

LAB1_PASS logic_gate cases=4와 40ns의 \$finish를 확인한다.
기본 실행 요청이 1000ns여도 테스트벤치가 40ns에서 종료한다.
파형만 열렸다는 사실로 통과 판정을 하지 않는다.

대상 보드와 핀 확인

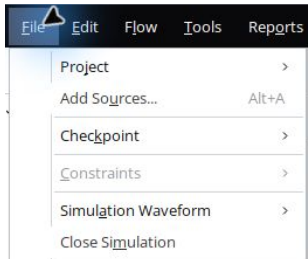
Constraints → logic_gate.xdc. 이번 원본 장비의 매핑이다.

포트	PACKAGE_PIN	역할
a	K4	SW1
b	N8	SW2
x	L4	LED1 / AND
y	M4	LED2 / OR
z	M2	LED3 / XOR

IOSTANDARD는 모두 LVCMOS33. 원본 장비 XDC와 대조했다.
다른 보드의 핀 번호로 바꾸지 않는다. 이 회로는 클록이 없는 조합회로다.

시뮬레이션 닫기

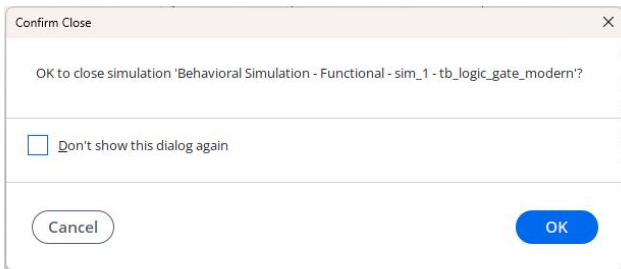
결과를 저장했으면 File → Close Simulation.



프로젝트 자체를 닫는 File → Project → Close와 구분한다.
이후 Project Manager에서 합성과 구현을 진행한다.

종료 확인 대화상자

달을 시뮬레이션 이름을 확인하고 OK.



검사 로그와 VCD는 프로젝트 폴더에 남는다.
이 화면을 지나면 Flow Navigator의 Run Synthesis를 선택한다.

합성 시작

Flow Navigator → Synthesis → Run Synthesis.

✓ Synthesis

▶ Run Synthesis

> Open Synthesized Design

Verilog 설계를 FPGA 자원으로 변환하는 단계다.
테스트벤치가 아닌 logic_gate를 설계 top으로 선택했는지 확인한다.

합성 Launch Runs 설정

Launch directory는 기본값, Launch runs on local host를 선택한다.

Launch the selected synthesis or implementation runs.

Launch directory

<Default Launch Directory>

Options

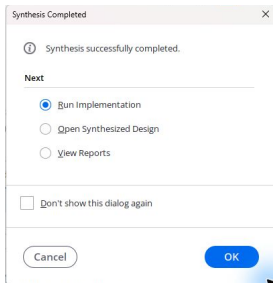
- ☒ Launch runs on local host: Number of jobs: 4
- ☐ Generate scripts only

OK

Number of jobs는 이 PC에서 4로 실행했다. PC 자원에 맞게 선택한다.
Generate scripts only 대신 실제 실행을 선택하고 OK. 완료를 기다린다.

합성 완료 후 구현 선택

Synthesis successfully completed를 확인한다.



Run Implementation을 선택하고 OK.
구현은 합성된 회로를 FPGA 안에 배치하고 배선한다.

구현 Launch Runs 설정

Launch runs on local host와 작업 수를 확인한다.

Launch the selected synthesis or implementation runs.

Launch directory

Options

☒ Launch runs on local host: Number of jobs:

☐ Generate scripts only



기본 Launch directory를 유지하고 OK.

완료 창이 나올 때까지 기다리며 실패하면 Messages를 확인한다.

구현 완료 후 비트스트림 선택

Implementation successfully completed를 확인한다.

 Implementation successfully completed.

Next

- ☐ Open Implemented Design
- ☒ Generate Bitstream
- ☐ View Reports

OK

Generate Bitstream을 선택하고 OK.

완료 창을 닫았다면 왼쪽 Program and Debug → Generate Bitstream.

비트스트림 Launch Runs 설정

비트스트림 생성 실행 창에서 설정을 확인한다.

Launch the selected synthesis or implementation runs and generate bitstream.

Launch directory

<Default Launch Directory>

Options

☒ Launch runs on local host: Number of jobs: 4

☐ Generate scripts only

OK

Launch runs on local host, 기본 디렉터리와 작업 수를 확인하고 OK.
write_bitstream이 완료될 때까지 기다린다.

비트스트림 생성 완료 확인

Bitstream Generation successfully completed가 성공 기준이다.



Bitstream Generation successfully completed.

Name	Constraints	Status
✓ synth_1	constrs_1	synth_design Complete!
✓ impl_1	constrs_1	write_bitstream Complete!

Design Runs의 impl_1은 write_bitstream Complete!로 바뀐다.
이 화면은 파일 생성 성공이며 실제 FPGA 기록 결과는 다음 단계다.

생성 파일과 경고 확인

프로젝트 폴더에서 생성한 logic_gate.bit를 확인한다.

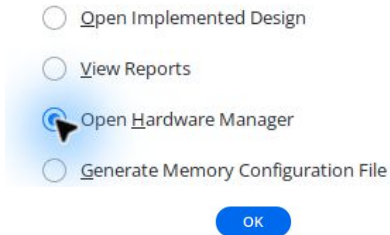
파일: vivado/logic_gate.runs/impl_1/logic_gate.bit.

이 실행의 DRC 오류는 0건이다. CFGBVS-1 경고는 보드의 구성 전압 확인이 필요하다.

클록 없는 조합회로의 타이밍 NA를 클록 회로의 통과로 해석하지 않는다.
Messages의 경고와 실행 로그를 실행 후 레포트에 함께 보관한다.

Hardware Manager로 이동

보드 실험은 Open Hardware Manager를 선택하고 OK.

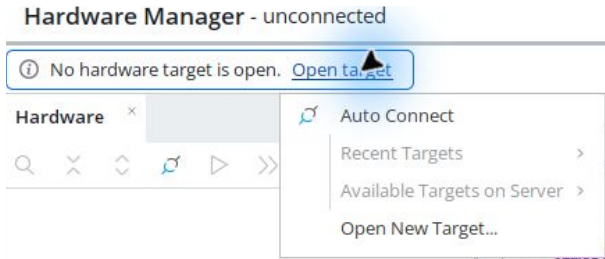


전원과 USB JTAG 연결을 확인한다.

완료 창을 닫았다면 Flow Navigator → Open Hardware Manager.

Open target → Auto Connect

Hardware Manager 위쪽 Open target을 누른다.



Auto Connect를 선택하고 연결된 장치 목록을 기다린다.
장치가 없으면 전원·USB 케이블·JTAG 드라이버·VM USB 연결을 점검한다.

장치에 기록할 때 확인할 것

장치가 연결되면 이름이 xc7s75인지 먼저 확인한다.

대상 장치를 우클릭 → Program Device.

Bitstream file에서 이번 프로젝트의 logic_gate.bit를 선택한다.

Program을 누르고 진행 완료와 장치 상태를 확인한다.

현재 제작 환경에서 장치 기록을 수행했는지는 검증표에 별도로 남긴다.

스위치·LED 사진과 시연 영상

a와 b를 00 → 01 → 10 → 11로 바꾸며 확인한다.

LED1 AND: 0,0,0,1 / LED2 OR: 0,1,1,1 / LED3 XOR: 0,1,1,0.
사진에는 보드 전체 연결과 스위치·LED의 위치가 함께 보여야 한다.
영상은 입력을 바꾸는 손과 출력 변화를 한 화면에서 보여준다.
교재 사진이 아닌 자신의 실제 실험 결과를 기록한다.

GitHub에 실행 결과 연결

자신의 저장소에 소스와 결과 자료를 올린다.

reports/pre, reports/post에 레포트를 두고 evidence/vscode, vivado, board로 구분한다.

사진·영상 파일명에는 회로 번호와 입력 또는 모드를 적는다.

큰 영상은 GitHub Release 등 배포 위치를 정하고 레포트에서 링크한다.
업로드 후 웹에서 사진 표시·영상 재생 또는 다운로드를 직접 확인한다.

실험 후 레포트의 비교와 해석

예상값 → VS Code → Vivado → 실제 보드를 비교한다.

Vivado 버전·part·top·XDC·생성 bit·실행 소스 커밋을 기록한다.
두 시뮬레이션의 파형, 합성·구현 결과와 경고를 해석한다.
장치 기록 화면·입출력 사진·시연 영상의 링크를 함께 붙인다.
미수행 항목은 미완료로 표시하고 원인과 다음 확인 순서를 적는다.

실습 완료 점검

1. 템플릿 clone부터 workspace 열기까지 직접 실행했는가?
2. VS Code에서 파일·작업·로그·파형을 직접 확인했는가?
3. 두 시뮬레이션의 같은 코드·네 입력·기대값이 연결되는가?
4. Vivado의 소스 종류와 top, 핀, 비트스트림 결과가 구분되는가?

사전 레포트에 자신의 실행 결과와 해석을 정리한다.

실험 후 레포트에는 Vivado 결과와 실제 보드 기록·사진·영상·GitHub 링크를 추가한다.

▶ 첫 회로 소스·workspace·검증 안내